What is Elasticsearch?

- A distributed search and analytics engine
- Stores data as JSON documents inside indexes
- Builds an inverted index: word → documents
- Everything is available over a REST API
- Fast, near real-time, scales across nodes

```
{
  "from_user": "Bob",
  "to_user": "John",
  "text": "I can't get into my account,
the password reset email never
arrives.",
  "text_sayt": "…same text…",
  "embedding": [0.012, -0.044, … ]
}
```

In this app, one chat message = one document in the messages index

Keyword search (BM25)

Stored message: “I can't get into my account, the password reset email never arrives.”

1 Analyze
Lowercase, split on spaces and punctuation

→

2 Tokens
account · password · reset · email · arrives …

→

3 Inverted index
password → message 42, message 57

→

4 BM25 score
Rarer, more frequent terms rank higher

Query: account ✓
The word “account” is in the index, so the message matches.

Query: login trouble ✗
Neither word appears in the message, so no results.

Search as you type



When the message is saved

- Field text_sayt has type search_as_you_type
- The text is analyzed into edge n-grams: p, pa, pas, pass … password
- Word pairs (2- and 3-word shingles) are stored too

When you type

- Your input goes through the same analyzer
- bool_prefix query: earlier words match fully, the last word as a prefix
- Runs on every keystroke (150 ms pause)

“pas” finds “password” — no full word needed

Semantic (kNN vector) search

1 Embed on send

Each message goes to OpenAI text-embedding-3-small and becomes 1536 numbers, saved in the embedding field.

2 Embed the query

“login trouble” is turned into a vector with the same model.

3 kNN + cosine

Elasticsearch finds the nearest vectors and drops anything below 0.35 similarity.

login trouble ≈ “I can't get into my account, the password reset email never arrives.”

No shared words, but the same meaning — so the two vectors are close together. An unrelated message like “hi” scores lower and is filtered out.